

The QueryGraph Stack

Stack release 0.4.0-86d0ce0 · built 2026-07-04

The Definitive Guide to the Governed Semantic Lakehouse

Alexy Khrabrov and Slava Tykhonov

querygraph.ai

Contents

Executive Summary	2
Links	3
The Stack at a Glance	3
Versions and Codenames	4
Grust: The Property-Graph Substrate	4
Worked example: build a graph, store it, query it	4
TypeSec: Security in the Type System	5
Worked example: a capability is the proof	6
Worked example: one seed, one identity, two languages	6
LakeCat: The Catalog Boundary	6
Worked example: from catalog to governed import	7
Sail and the Cypher Extension	7
QueryGraph in Rust: The Governed Semantic Layer	8
Worked example: Croissant becomes OSI, identically in both languages	8
QueryGraph in Python: The Pythonic Mirror	9
Worked example: the navigator loop, receipts and all	9
The Evidence Chain, End to End	10
Worked example: sign in Python, verify in Rust	10
Interoperability Surfaces	11
Worked example: the guarded /v1, from both sides	11
Worked example: an MCP session, by hand	12
The Cross-Language Contract	12
Operating the Stack	12
Roadmap	13
Link Index	13

Executive Summary

The QueryGraph stack is a governed semantic lakehouse for agentic AI: a set of coordinated open-source components that let AI agents answer questions over enterprise data while *proving* what they did — which semantics they used, which policies allowed it, which sources they touched, and which they were denied. The stack’s thesis is that the differentiating infrastructure for enterprise AI is not a bigger model or a longer context window, but a verifiable chain from question to answer: deterministic identity, dual policy gating, signed envelopes, canonical hashing, and lineage anchoring, all projected through open standards.

Five components make up the stack, each independently useful, released in coordinated, codenamed versions:

- **Grust** — a backend-neutral property-graph API for Rust with a GQL/Cypher read surface and a dozen storage backends. The semantic graph substrate.
- **TypeSec** — agentic AI security in Rust’s type system: unforgeable capabilities, TypeDID signed agent envelopes, audit events. The security fabric.
- **LakeCat** — a Rust-native Apache Iceberg REST catalog that binds catalog state, governed Sail planning, TypeSec receipts, and Grust projection to the same table transitions. The catalog boundary.
- **Sail** (fork) — the Spark-compatible compute engine, extended with a Cypher graph-query surface compiled into its SQL frontend. The lakehouse engine.
- **QueryGraph** — the semantic layer itself, in Rust (qg-rust) and Python (qg-python): four semantic projections (Semantic Croissant, CDIF, W3C DID, ODRL) plus OSI business semantics, RBAC+ODRL dual gating, Ed25519-signed TypeDID envelopes verified across languages, OpenLineage audit with signed attestations, an HTTP /v1 API, MCP servers in both languages, and an A2A agent card.

As of this guide, the current releases are **QueryGraph 0.4.0 “Sentinel”** (the governed-answer release, in both languages — the navigator loop, envelope authentication on /v1, and a dependency-free Rust MCP server, following 0.3.0 “Goshawk”, the interoperability release), over **Grust 0.12.0 “Lobster”**, **TypeSec 0.12.0 “Torcello”**, and **LakeCat 0.3.0 “Ocelot”** — a coordinated substrate wave in which Grust merged its Full39075 GQL goal, TypeSec became an agent-interoperability security platform, and LakeCat proved stock-client Iceberg REST conformance.

Links

What	Where
Workspace meta-repo (map, review, this guide's home)	https://github.com/querygraph/querygraph
QueryGraph Rust (qg-rust)	https://github.com/querygraph/qg-rust
QueryGraph Python (qg-python)	https://github.com/querygraph/qg-python
Grust	https://github.com/querygraph/grust
TypeSec	https://github.com/querygraph/typesec
LakeCat	https://github.com/querygraph/lakecat
Sail upstream (forked with the Cypher extension)	https://github.com/lakehq/sail
Sentinel releases	https://github.com/querygraph/qg-rust/releases/tag/v0.4.0 · https://github.com/querygraph/qg-python/releases/tag/v0.4.0
The dedicated QueryGraph book	qg-rust/docs/book
Semantic Croissant (MLCommons Croissant)	https://mlcommons.org/croissant/
CDIF (CODATA Cross-Domain Interoperability Framework)	https://cdif.codata.org/
W3C DID / ODRL	https://www.w3.org/TR/did-core/ · https://www.w3.org/TR/odrl-model/
OSI (Open Semantic Interchange)	https://github.com/open-semantic-interchange
OpenLineage	https://openlineage.io/
Model Context Protocol	https://modelcontextprotocol.io/
Agent2Agent (A2A)	https://github.com/a2aproject/A2A

This guide is the stack-wide companion to the dedicated QueryGraph book (which walks the semantic layer itself as a textbook, layer by layer). Read this guide to understand how the five components fit together, what each guarantees to the others, and how to operate the whole; read the dedicated book for the deep treatment of the semantic projections and the governed agent story. Each component chapter ends with worked examples — Rust and Python as applicable, with outputs captured from real runs against the released code.

The Stack at a Glance

A question enters the stack at the top and evidence comes out at the bottom:

1. An agent (or a person, or another agent framework over MCP or /v1) asks a question.
2. QueryGraph searches the **semantic model** — OSI business terms, Croissant field semantics, ontology terms — for what the question means here.
3. The **rights layer** decides, per source, whether this principal may read it: RBAC *and* ODRL must both allow. Every decision, allow or deny, becomes a receipt.
4. SQL is planned only over allowed **Sail** sources; denied sources are named as off-limits.
5. Synthesis happens — deterministic, or via any LLM — and the answer travels in a **TypeDID envelope** signed with Ed25519 under a did:key verification method.
6. An **OpenLineage** event (spec-conformant, schema-validated) records the run, and an Ed25519 **attestation** anchors the event hash.

Each step is somebody's component. The semantic graph lives in Grust (and can be queried through Sail's Cypher extension). The envelope machinery and capability model are TypeSec. The tables come from a lakehouse whose catalog transitions LakeCat governs. And QueryGraph is the layer that makes them one system with one evidence chain.

Versions and Codenames

Stack releases are coordinated by codename, each repo keeping SemVer independently:

Component	Version	Codename	Pool
QueryGraph (both languages)	0.4.0	Sentinel	birds of prey
Grust	0.12.0	Lobster	—
TypeSec	0.12.0	Torcello	Venetian landmarks
LakeCat	0.3.0	Ocelot	wild cats

The 0.12 substrate wave landed together: Grust merged the Full39075 GQL profile, TypeSec shipped its agent-interoperability platform, and LakeCat moved to both while proving stock-client Iceberg REST conformance — with QueryGraph verified green against all three. QueryGraph 0.4.0 “Sentinel” ships on top of that wave.

Grust: The Property-Graph Substrate

Grust is a modern property-graph API for Rust with a deliberately plain core model — Graph = nodes + edges, nodes and edges carrying ids, labels, and typed properties — and a strict separation between graph construction and database query languages. Application code builds a `grust::Graph`; backend crates decide how to persist or query it.

The workspace ships a facade crate (`grust-graph`) over a core (model, builder, schema, traversal IR, `GraphStore` trait) and backends for, among others: deterministic in-memory storage, SurrealDB, HelixDB, FalkorDB, LadybugDB, LanceDB, PostgreSQL (generic tables, SQL/PGQ, `pgGraph`), Turso, and — the one QueryGraph leans on — **grust-sail**, a Sail SparkConnect backend that stores graphs as Spark DataFrames.

On the read side, `grust-cypher` provides a Cypher/GQL surface over any store. QueryGraph uses Grust twice: the semantic graph (datasets, fields, ontology terms, agents, policies as nodes and edges) loads into a Grust store, and the Sail fork compiles a Cypher extension *into* the engine so the same graph is queryable from Spark Connect sessions.

The Lobster release (0.12.0) completes the query language: the merged “Full39075” GQL profile — CALL { ... } subqueries, table-valued functions, `shortestPath()/allShortestPaths()`, backend-native passthrough escape hatches — with an executable portable read corpus, and atomic Cypher transaction batches behind the transaction surface.

Worked example: build a graph, store it, query it

Graph construction is backend-neutral — the same `GraphBuilder` code feeds memory, SurrealDB, PostgreSQL, or Sail:

```
use grust::prelude::*;

let mut builder = GraphBuilder::new();
let dataset = builder
    .node("Dataset", "dataset:county-finance")
    .prop("source", "sail.qg_lakehouse.government_finance__countydata")
    .finish();
```

```

let metric = builder
  .node("Metric", "metric:fiscal-capacity")
  .prop("expression", "SUM(total_revenue - mandated_spend)")
  .finish();
builder.edge("MEASURED_OVER", &metric, &dataset).finish();
let graph = builder.build();

// Persist and traverse (memory store; SailGraphStore is the same trait).
let store = MemoryGraphStore::new();
store.put_graph(&graph).await?;
let datasets = store
  .traverse(
    Traversal::from_node("metric:fiscal-capacity")
      .out("MEASURED_OVER")
      .to("Dataset"),
  )
  .await?;
assert_eq!(datasets.len(), 1);

```

The same graph answers Cypher through grust-cypher's portable read executor — exactly how qg-rust queries its semantic graph (src/cypher.rs):

```

use grust_cypher::read::run_read_query;
use grust_cypher::CypherParameters;

let table = run_read_query(
  &graph,
  "MATCH (m:Metric)-[:MEASURED_OVER]->(d:Dataset) \
  RETURN m.expression, d.source",
  &CypherParameters::new(),
)?;

```

Through the Sail fork's Cypher extension, that same MATCH also runs from any Spark Connect session against the graph the lakehouse projects.

TypeSec: Security in the Type System

TypeSec encodes permissions as types. A `Capability<CanWrite, Report>` is an unforgeable proof: its constructor is crate-private, its permission and resource parameters are phantom types, and the only production path to one runs a policy check and emits an audit event. If your code does not hold the capability, the guarded function does not exist for you. Violations are compile errors, not runtime surprises.

On top of the capability core, TypeSec provides the **TypeDID** machinery QueryGraph uses for agent identity and messaging: Ed25519 DID keys derived deterministically from seeds, did:key documents, and signed, encrypted agent envelopes (ed25519-x25519-chacha20 profile) whose audit-safe attestations expose who did what to which resource at which privacy level — without revealing the payload. QueryGraph wraps every agent request and response in these envelopes; the supervised multi-agent story mints capabilities for delegation, sensitive reads, and summary aggregation.

The Torcello release (0.12.0) — TypeSec's own agent-interoperability release — runs remarkably parallel to QueryGraph's Goshawk: an agent-framework interop plane that guards OpenAI, Anthropic, LangChain, and Pydantic-AI tool calls; an MCP dialect and mcp-gate, a deny-by-default MCP stdio proxy; signed decision receipts with decision logging and replay; JSON-Schema-validated tool bindings; a PyPI-ready Python package; a WASM decision core; and an OpenAI/Anthropic-

compatible **enforcement proxy** — the “governed inference proxy” pattern, built at the security layer. QueryGraph already builds against Torcello; adopting these new surfaces rather than duplicating them is the next integration step.

Worked example: a capability is the proof

There is no `if acl.check(...)` to forget. The guarded function demands the capability type, and the only production path to a capability runs the policy engine and emits an audit event:

```
use typesec::prelude::*;

// Without a Capability<CanRead, Report> in hand, this function
// cannot be called — the check is the type system's, not yours.
fn read_report(cap: Capability<CanRead, Report>, report: &Report) -> Summary {
    summarize(report) // `cap` in scope == the policy engine said yes
}

let decision = engine.check(&subject, Action::Read, &report);
match decision {
    Ok(cap) => read_report(cap, &report),      // audit event already emitted
    Err(denied) => return Err(denied.receivept()) // a denial is data, not panic
};
```

Worked example: one seed, one identity, two languages

QueryGraph agents derive Ed25519 keys from seeds the way TypeSec’s `Ed25519DidKey::from_seed` does — so the same seed yields the same `did:key` in Python and Rust, and each side verifies what the other signs.

Python signs:

```
from querygraph.typedid import TypeDidAgent

supervisor = TypeDidAgent.new("SupervisorAgent")
envelope = supervisor.request(
    TypeDidAgent.new("FinanceAgent"),
    action="summarize", resource="compartment:finance",
    payload={"question": "Where is fiscal stress highest?"},
)
print(envelope.signature[:24])      # ed25519:8a7a231b6f67f4...
print(envelope.verification_method[:32]) # did:key:z6Mkrdhp...
```

Rust mints the identical identity from the identical seed:

```
use querygraph::agent::PyTypeDidEnvelope;

let envelope = PyTypeDidEnvelope::signed(
    "querygraph-agent:SupervisorAgent", // same seed => same did:key
    "did:example:recipient",
    "summarize", "compartment:finance",
    serde_json::json!({"question": "Where is fiscal stress highest?"}),
);
assert!(envelope.verify().signature_valid);
```

A fixture test pins the shared `did:key` so the derivations can never drift.

LakeCat: The Catalog Boundary

LakeCat is a Rust-native Apache Iceberg REST catalog and QueryGraph foundation. Standard Iceberg clients speak to it on ordinary REST catalog paths (`/catalog/v1`); underneath, it binds

catalog state, governed Sail planning, TypeSec receipts, and Grust projection to the same accepted table transition, so the catalog's view of a table and the governance evidence about it can never drift apart.

For QueryGraph, LakeCat's signature surface is the **bootstrap bundle** at `/querygraph/v1/bootstrap`: a projection of live catalog tables into Croissant, CDIF, OSI, ODRL, OpenLineage, and a Grust-ready graph envelope. The wire format lives in a shared crate (`qglake-bundle`), so `qg-rust` verifies and imports bundles without copying formats. Scan planning routes through the Sail-facing engine, with point-in-time and append-only incremental scans producing Iceberg REST plan-task tokens from stable Sail metadata.

The Ocelot release (0.3.0) proves the boundary from the stock client's side: Iceberg REST conformance demonstrated by a PyIceberg round-trip — spec-correct error types (403 on authorization denial, 409 on a duplicate namespace, 404 on a missing one), `listTables`, honest update rejection on the default build, and fail-closed commit-requirement validation — over dependencies moved to Grust "Lobster" and TypeSec "Torcello". The release rests on a recorded release-candidate proof: a full local handoff harness that creates a fixture, plans through Sail, writes Turso catalog and Grust graph state, verifies replay and OpenLineage evidence, and runs QueryGraph's locked `verify/import` commands, 40/40 green.

Worked example: from catalog to governed import

Run LakeCat with its local integrations, then hand its bootstrap bundle to QueryGraph:

```
# Terminal 1: the catalog, with Sail planning, TypeSec receipts,
# and Grust projection bound to table transitions.
LAKECAT_BIND_ADDR=127.0.0.1:8181 \
LAKECAT_TURSO_PATH=target/local/catalog.turso \
LAKECAT_GRUST_TURSO_PATH=target/local/catalog-graph.turso \
cargo run -p lakecat-service \
  --features sail-local,typesec-local,grust-turso-local,turso-local

# Terminal 2: standard Iceberg clients see an ordinary REST catalog...
cargo run -p lakecat-cli -- config --catalog http://127.0.0.1:8181

# ...and QueryGraph sees live tables projected into Croissant, CDIF,
# OSI, ODRL, OpenLineage, and a Grust-ready graph envelope.
curl -s http://127.0.0.1:8181/querygraph/v1/bootstrap > bootstrap.json

# qg-rust verifies the bundle with LakeCat's own shared wire crate
# (qglake-bundle) and emits its import plan:
cargo run -- lakecat-import --bundle bootstrap.json --output import-plan.json
```

The import command prints the bundle's verification report; QueryGraph accepts catalog state as *proof*, not as a best-effort side effect.

Sail and the Cypher Extension

Sail is the Spark-compatible lakehouse engine (from lakehq) that QueryGraph uses as its compute and storage substrate: typed Parquet tables registered in a Spark Connect-compatible server, queryable from PySpark, with the QueryGraph audit trail (`qg_audit`) living alongside the data.

The stack's fork adds a **Cypher graph-query extension** compiled into Sail itself — roughly 5,600 lines across the SQL parser (Cypher AST), analyzer (graph analysis), and plan resolver, reusing Grust's property-graph model and schema validation rather than reimplementing them. The result:

the same semantic graph that QueryGraph loads through Grust is MATCH-able from any Spark Connect session, no separate graph database required.

QueryGraph in Rust: The Governed Semantic Layer

qg-rust is the reference implementation. Its layers:

- **Semantic projections.** Every dataset is described four ways: Semantic Croissant (ML-ready dataset metadata, JSON-LD), CDIF (discovery, access, and profile projection), W3C DID (deterministic did:oyd identity documents), and ODRL (permissions and prohibitions as policy). OSI models project business terms — datasets, fields, metrics with per-dialect SQL expressions, ontology terms — over Croissant fields and governed Sail columns.
- **Governance.** RBAC roles and ODRL policies gate together — both must allow. Decisions are receipts. TypeDID envelopes carry agent requests and responses with real signatures.
- **Lakehouse.** Loaders stream CSV/TSV/XLSX into typed Parquet in Sail, register views, and keep a manifest; the dataverse-e2e path takes live Dataverse datasets through the whole chain, optionally calling Ollama through a DID-encrypted prompt gateway.
- **Lineage.** OpenLineage COMPLETE events (spec-conformant UUIDv5 run ids, validated against the official 2-0-2 JSON Schema), canonical event hashing, Ed25519-anchored attestations, JSONL and Sail sinks.
- **Interfaces.** A CLI (navigator bundles, the QGLake story, lakehouse loading, verification); the /v1 HTTP API (serve); an MCP stdio server (mcp-serve); the A2A agent card (agent-card, also served at /.well-known/agent-card.json); and verify-envelope, which verifies qg-python's Ed25519 envelopes with no shared state.

The **QGLake story** — the Resilience Desk — is the governed multi-agent narrative both books use: a supervisor delegates to compartmentalized specialists (finance, energy, mobility, climate-health, reference), a restricted-data broker returns a signed denial instead of raw rows, synthesis sees only signed summaries, and the whole run emits an OpenLineage event anchored by an Ed25519 attestation. It is deterministic by design: the golden baseline the live navigator loop is measured against.

Worked example: Croissant becomes OSI, identically in both languages

A Semantic Croissant document projects into an OSI model — every recordSet field becomes a governed SAIL_SQL column expression, sameAs semantic types become ontology terms, and a row_count metric is attached. In Rust (OsiDocument::from_croissant_json, the code behind POST /v1/models/import/croissant):

```
let croissant = serde_json::json!({
  "name": "Energy Burden",
  "description": "Demo energy fields",
  "recordSet": [{ "field": [{
    "name": "monthly_cost",
    "description": "Monthly household energy cost",
    "sameAs": "https://querygraph.ai/ontology/monthlyEnergyCost",
  }]}],
});
let osi = OsiDocument::from_croissant_json(&croissant, "qg_lakehouse")?;
assert_eq!(osi.semantic_model.name, "energy_burden_semantic_model");
assert_eq!(osi.semantic_model.datasets[0].source,
  "sail.qg_lakehouse.energy_burden");
```

And in Python, where the enriched model adds dialect-fallback metric resolution and synonym search:

```

from querygraph.osi import OsiDocument

osi = OsiDocument.from_croissant(dataset)           # same projection rules
osi.semantic_model.resolve_metric("row_count")      # 'COUNT(*)' via SAIL_SQL
osi.semantic_model.find_by_synonym("energy")        # datasets/fields/metrics

```

QueryGraph in Python: The Pythonic Mirror

qg-python mirrors the same layers Pydantic-v2-first, and adds the ecosystem surfaces Python is for:

- **Real crypto** (crypto extra): Ed25519 signing with seed-derived keys (sha256(seed) as private key, exactly TypeSec's derivation), did:key verification methods, envelope and attestation verification. Without the extra, digests are labeled unsigned:sha256: — never mistakable for signatures.
- **The governed navigator loop** (querygraph answer): question → semantic search (names, synonyms, bigrams) → RBAC+ODRL receipts → SQL plans over allowed sources → synthesis via any Callable[[str], str] (openai_compatible_llm binds Ollama, vLLM, llama.cpp, LM Studio, OpenRouter) or the deterministic baseline → signed envelope + schema-valid OpenLineage + attestation.
- **MCP server** (mcp extra, querygraph mcp-serve): search, metric resolution with dialect fallback, access checks whose denials are receipts, bundle building, the story, envelope verification, and answer_question.
- **Framework adapters**: LangChain StructuredTool (sync and async), vendor-neutral TypeDidAgent.to_tool_schema() in OpenAI and Anthropic flavors, and api_auth for minting /v1 envelope-auth headers.
- **Lakehouse helpers**: PySpark against Sail's Spark Connect endpoint, registering the loaded tables and audit views.

Worked example: the navigator loop, receipts and all

```

from querygraph.navigator_loop import GovernedNavigatorLoop

loop = GovernedNavigatorLoop.demo()  # or (your_osi_doc, your_rights, llm=...)
result = loop.answer(
    "Where do fiscal capacity and energy burden overlap with health risk?"
)

```

The result — captured from a real run — plans only over allowed sources and carries the denial as a first-class receipt:

```

{
  "answer": "Answerable from governed sources
    sail.qg_lakehouse.access_2018__access_data,
    sail.qg_lakehouse.government_finance__countydata via 3 planned
    queries. Restricted sources were denied with receipts:
    sail.qg_lakehouse.haalsi_baseline__restricted_raw.",
  "denied_sources": ["sail.qg_lakehouse.haalsi_baseline__restricted_raw"],
  "plans": [
    "SELECT * FROM sail.qg_lakehouse.government_finance__countydata",
    "SELECT `monthly_cost` FROM sail.qg_lakehouse.access_2018__access_data",
    "SELECT SUM(total_revenue - mandated_spend)
      FROM sail.qg_lakehouse.government_finance__countydata"
  ]
}

```

The denied source's receipt names the principal, the action, and the policy that refused it:

```
{
  "principal": "did:example:qg-navigator",
  "resource": "sail.qg_lakehouse.haalsi_baseline__restricted_raw",
  "action": "odrl:read",
  "allowed": false,
  "reason": "RBAC or ODRL denied action",
  "policy_id": "urn:querygraph:policy:navigator-demo"
}
```

Swap in a live model with one callable — the governance is unchanged:

```
from querygraph.navigator_loop import openai_compatible_llm

loop = GovernedNavigatorLoop.demo(
    llm=openai_compatible_llm("http://localhost:11434", "llama3.2"),
    llm_name="ollama:llama3.2",
)
```

The Evidence Chain, End to End

The stack's product is the chain, not any single hop:

1. **Identity** is deterministic. Agents derive `did:oyd` documents from seeds; signing keys derive from the same seeds; the same seed yields the same identity in Rust and Python.
2. **Policy** is dual-gated. RBAC says the role may act on the resource; ODRL says the policy permits the action and nothing prohibits it. Only both together allow. Every decision is a receipt with a reason.
3. **Messages** are signed envelopes. Payloads are canonically hashed (`sort_keys`, `compact`, `ensure_ascii`); signatures cover a documented payload (`querygraph-typedid-signing-v1`); verification methods are `did:key` identifiers anyone can resolve.
4. **Lineage** is standard. Runs emit OpenLineage events with deterministic UUIDv5 run ids under a shared namespace, validated against the official schema — the same seed produces the same run id in either language.
5. **Anchoring** is cryptographic. Event hashes are signed into attestations; the audit trail lands in JSONL and in Sail tables next to the data.

A useful way to read the chain: *the answer used OSI metric X, resolved to Sail table Y, under capability C and `odrl:read`, emitting OpenLineage run R anchored by attestation A — and source Z was denied, with a receipt*. No mainstream agent framework offers that sentence.

Worked example: sign in Python, verify in Rust

```
# Python signs an envelope and writes it out...
uv run python - <<'PY'
import json
from querygraph.typedid import TypeDidAgent
env = TypeDidAgent.new("SupervisorAgent").request(
    TypeDidAgent.new("FinanceAgent"),
    action="summarize", resource="compartment:finance",
    payload={"question": "Where is fiscal stress highest?"})
open("envelope.json", "w").write(env.model_dump_json())
PY

# ...and the Rust CLI verifies it with no shared state:
cargo run -- verify-envelope --file envelope.json
```

```
{
  "payload_hash_valid": true,
  "signed": true,
  "signature_valid": true,
  "verification_method": "did:key:z6MkrdhpFnCtEGK3RhXqryfjxVpy...",
  "scheme": "ed25519"
}
```

Change one byte — the resource, a payload key, a signature hex digit — and `signature_valid` flips to `false` and the command exits non-zero. The same check is served at `POST /v1/audit/verify-envelope` and as the `verify_envelope` MCP tool in both languages.

Interoperability Surfaces

Goshawk's organizing principle: QueryGraph does not compete with agent frameworks — it is the governed data and semantics plane they plug into.

- **/v1 HTTP API** (Rust, axum): health, navigator bundles, the story, envelope audit, a semantic-model registry (import OSI or Croissant, list, fetch, search), and answer. With `--require-auth`, governed routes demand a signed envelope in `x-qg-envelope`, action invoke, resource bound to the request path, payload bound to the body hash — no cross-route replay, no body swapping. Denials are 401 receipts that teach the contract.
- **MCP** in both languages: one server reaches Claude, LangChain, PydanticAI, LlamaIndex, CrewAI, and the OpenAI Agents SDK with zero per-framework code. The Rust implementation is dependency-free JSON-RPC over stdio; the Python one uses the official SDK and adds `qg://resources`.
- **A2A**: both languages publish the same Agent2Agent card (five skills, a TypedID security scheme) at `/.well-known/agent-card.json` and via `agent-card`; the equivalence suite asserts the cards agree.
- **Tool schemas**: `to_tool_schema()` exports OpenAI- and Anthropic-flavor JSON-Schema tool definitions accepted by most runtimes and local servers.
- **LLM providers**: the loop binds any OpenAI-compatible endpoint; the Rust Ollama path wraps calls in DID-encrypted prompt-bound envelopes.

Worked example: the guarded /v1, from both sides

An unauthenticated request to a governed route gets a 401 whose receipt *teaches the contract* (captured from a live server):

```
$ curl -s -X POST http://127.0.0.1:8080/v1/answer \
  -H 'content-type: application/json' -d '{"question":"?"}'

{
  "error": "missing x-qg-envelope header",
  "receipt": {
    "allowed": false,
    "path": "/v1/answer",
    "contract": {
      "header": "x-qg-envelope",
      "action": "invoke",
      "resource": "<request path>",
      "payload": {"bodySha256": "<sha256 hex of request body>"},
      "signature": "ed25519 over querygraph-typedid-signing-v1"
    }
  }
}
```

The Python client satisfies it in two lines — the envelope is bound to this path and this body, so it can be neither replayed elsewhere nor reattached:

```
from querygraph.api_auth import governed_post
from querygraph.typedid import TypeDidAgent

result = governed_post(
    "http://127.0.0.1:8080", "/v1/answer",
    {"question": "what is fiscal capacity?"},
    TypeDidAgent.new("ApiClient"),
)
```

Worked example: an MCP session, by hand

The Rust server speaks MCP over stdio with zero dependencies — here is a full session, three lines in, structured answers out:

```
$ printf '%s\n%s\n%s\n' \
'{"jsonrpc":"2.0","id":1,"method":"initialize","params":{}}' \
'{"jsonrpc":"2.0","method":"notifications/initialized"}' \
'{"jsonrpc":"2.0","id":2,"method":"tools/list"}' \
| querygraph mcp-serve
{"id": 1, "result": {"protocolVersion": "2024-11-05",
  "serverInfo": {"name": "querygraph", "version": "0.4.0"}, ...}}
{"id": 2, "result": {"tools": ["build_navigator_bundle",
  "run_qglake_story", "verify_envelope", "import_semantic_model",
  "search_semantic_models", "answer_question"]}}
```

For function-calling runtimes instead of MCP, one agent exports both flavors:

```
agent = TypeDidAgent.new("FinanceAgent")
agent.to_tool_schema() # {"type": "function", "function":
# {"name": "FinanceAgent",
# "parameters": {...}} ← OpenAI
agent.to_tool_schema(flavor="anthropic") # {"name": "FinanceAgent",
# "input_schema": {...}} ← Anthropic
```

The Cross-Language Contract

Rust and Python are held equivalent by an executable contract (qg-python/tests/test_rust_equivalence.py), not by discipline:

- navigator bundles are byte-identical modulo timestamps.
- The QGLake stories agree on governance semantics: same specialist roster, the restricted broker (and only it) denied, attestation schemas field-for-field identical.
- Python signs an envelope; the Rust CLI verifies it and rejects a tampered copy with a non-zero exit. Rust mints envelopes from the same seeds and Python's key derivation matches exactly (a fixture pins the shared did:key).
- Both CLIs' OpenLineage events validate against the official schema; a fixture pins the shared UUIDv5 run-id derivation.
- Both agent cards publish identical skills, capabilities, and security schemes.
- A live test boots qg-rust serve --require-auth, posts with a Python-minted auth header (200), and without one (401 with receipt).

Operating the Stack

The workspace expects sibling checkouts:

```
~/src/
├─ querygraph/          # meta-repo: README, FABLE-REVIEW-1, this guide's sources
│   └─ qg-rust/         # reference implementation (this repo)
│       └─ qg-python/   # Pythonic mirror
│           └─ sail/    # fork, branch `grust`, Cypher extension
│               └─ semantic/ # research: Polaris SemanticModel, OSI round-trips
├─ grust/               # path dependency of qg-rust
├─ lakecat/            # path dependency of qg-rust
└─ typesec/           # consumed from crates.io
```

Quick start:

```
# Rust: tests, story, server, MCP
cd qg-rust && cargo test
cargo run -- qglake-story --json
cargo run -- serve --port 8080 --require-auth
cargo run -- mcp-serve

# Python: everything on
cd qg-python && uv sync --extra all
uv run pytest                # includes the cross-language contract
uv run querygraph answer --question "Where do fiscal and energy stress overlap?"
uv run querygraph mcp-serve --osi model.yaml
```

Both repos run GitHub Actions CI; qg-rust's assembles the sibling layout. Release engineering is disciplined: SemVer, codename pools, coordinated stack versions, CHANGELOGs and RELEASES logs per repo, versioned EPUB/PDF book artifacts, and GitHub releases with attached wheels.

Roadmap

The near line (post-Sentinel):

- the navigator loop maturing from deterministic baseline to live LLM runs under identical receipts, and its Rust parity;
- the remaining /v1 surface (lineage event queries, audit verification, access explanation) behind envelope auth;
- adopting TypeSec Torcello's new surfaces: the interop plane, mcp-gate, signed decision receipts, and the enforcement proxy at the security layer — the dependency bump is done; the integration is the work.

The wider arc, from the workspace review (FABLE-REVIEW-1 in the meta-repo): Polaris SemanticModel entities and /navigator-bundle projection (LakeCat first, then upstream), OSIMetricFacet upstreaming to OpenLineage, dbt MetricFlow and Cube importers into OSI, Hugging Face Croissant importers, an ADBC/Flight SQL path for lightweight Python querying, Merkle-batched attestations, ontology import, cross-node federation — and the benchmark that motivates it all: measuring how much a governed semantic layer improves agent accuracy over the same lakehouse.

Link Index

- Meta-repo: <https://github.com/querygraph/querygraph>
- qg-rust: <https://github.com/querygraph/qg-rust> · release <https://github.com/querygraph/qg-rust/releases/tag/v0.3.0>
- qg-python: <https://github.com/querygraph/qg-python> · release <https://github.com/querygraph/qg-python/releases/tag/v0.3.0>
- Grust: <https://github.com/querygraph/grust>

- TypeSec: <https://github.com/querygraph/typesec>
- LakeCat: <https://github.com/querygraph/lakecat>
- Sail upstream: <https://github.com/lakehq/sail>
- Standards: Croissant <https://mlcommons.org/croissant/> · CDIF <https://cdif.codata.org/> · DID <https://www.w3.org/TR/did-core/> · ODRL <https://www.w3.org/TR/odrl-model/> · OSI <https://github.com/open-semantic-interchange> · OpenLineage <https://openlineage.io/> · MCP <https://modelcontextprotocol.io/> · A2A <https://github.com/a2aproject/A2A>